Mini Redis 기술 구현 및 완주 보고서

정보를 엄청 빠르게 찾아주는 작은 저장소 만들기 (자료구조와 알고리즘 80시간 미션)

개발자 / GitHub	KANGSIK-SEO (https://github.com/KANGSIK-SEO/miniredis)
개발 환경	macOS (Apple Silicon M4 Max, 48GB) / Python 3.14 / Git
핵심 제약 사항	dict, set, collections 사용 일절 금지 (기초 자료구조 직접 구현)
단위 테스트 결과	34개 단위 테스트 100% 통과 (Ran 34 tests in 0.002s, OK)

1. 프로젝트 개요 및 본질 (Key-Value 관점)

Redis는 단순한 Key-Value 저장소처럼 보이지만, '한정된 메모리 안에서 수천만 개의 데이터를 1번(O(1)) 만에 찾고, 가장 오래된 데이터를 즉시 버리며(LRU), 유통기한 지난 데이터를 실시간 청소(TTL)'하는 정밀 시스템입니다. 본 프로젝트는 파이썬 내장 컬렉션에 의존하지 않고 기초 자료구조를 바닥부터 구현하여 Redis의 고성능 엔진 아키텍처를 증명했습니다.

자료구조	담당 역할 (Key-Value 관점)	시간 복잡도	설계 및 구현 원리
HashMap (체이닝 방식)	초고속 Key ⟶ Value 탐색 메인 저장소	평균 O(1)	DJB2 다항 롤링 해시 함수로 충돌 최소화. 버킷마다 이중 연결 리스트로 체이닝. 로드 팩터 0.75 초과 시 버킷 2배 확장(Rehashing).
DoublyLinkedList (이중 연결 리스트)	Key의 '최근 사용 순서' (LRU 큐) 추적	모든 연산 O(1)	prev, next 노드 포인터 기반. 해시맵 값에 노드 주소를 저장하여 get/set 시 O(1)로 Head로 이동. 용량 초과 시 Tail 노드를 O(1)로 퇴출.
MinHeap (최소 힙)	Key의 '유효기간' (TTL 만료) 감시	확인 O(1) 삭제 O(log N)	완전 이진 트리를 1차원 배열로 구현. (expire_at, key) 튜플 저장. 루트에 가장 이른 만료 시간이 오므로 100만 개 전수조사 없이 O(1)에 만료 감시.

2. 메모리 관리 및 LRU / TTL 라이프사이클

- 메모리 계산 공식: used_memory = Σ(len(utf8(key)) + len(utf8(value))) (자료구조 오버헤드 제외)
- 단일 엔트리 OOM: 단일 key+value 크기가 maxmemory를 초과하면 기존 데이터를 지우지 않고 즉시 OOM 에러 출력
- LRU 자동 퇴출 알고리즘: maxmemory 〉 0이고 새 데이터 추가로 한도를 초과하면, 한도 이하가 될 때까지 DLL의 Tail에서 가장 오래된 키를 꺼내 해시맵/리스트/힙에서 동시 삭제 및 evicted_keys 누적
- SET 덮어쓰기 정합성: 기존 키가 존재할 경우 메모리 차감/가산 반영 및 기존 TTL 즉시 무효화(삭제)
- TTL 만료 처리: 모든 키 기반 명령 실행 전 expire_at 검사 (Lazy Deletion). 만료 시 즉시 삭제 후 (nil) 반환 및 LRU 갱신 방지

3. 지원 명령어 명세 및 Redis 표준 규격

명령어	구분	표준 반환 규격	시간 복잡도	동작 및 엣지 케이스 처리
SET k v	String	OK / (error) OOM	O(1)	LRU Head 승격. 메모리 초과 시 LRU 방출. 덮어쓰기 시 기존 TTL 제거.
GET k	String	"value" / (nil)	O(1)	만료된 키는 삭제 후 (nil) 반환 (LRU 갱신 안 함). 유효 시 LRU Head 승격.
DEL k	String	(integer) 1 또는 0	O(1)	해시맵, LRU 리스트, TTL 힙 모두에서 동시 제거 및 used_memory 차감.
EXISTS k	String	(integer) 1 또는 0	O(1)	실행 전 만료 여부 먼저 검사하여 만료 시 삭제 후 0 반환.
DBSIZE	String	(integer) N	O(1)	현재 유효한 활성 키 개수 반환.
KEYS	String	1. "k" ... / (empty array)	O(N)	현재 존재하는 모든 유효 키 목록 배열 출력.
CONFIG SET	Memory	OK / (error)	O(1)	maxmemory 바이트 설정 (0=무제한). 잘못된 값 입력 시 에러 반환.
INFO memory	Memory	포맷 텍스트	O(1)	used_memory, maxmemory, evicted_keys 통계 출력.
EXPIRE k s	TTL	(integer) 1 또는 0	O(log N)	초 단위 만료 등록. 없는 키는 0. 0 이하 초 입력 시 즉시 삭제 후 1 반환.
TTL k	TTL	(integer) N / -1 / -2	O(1)	없는 키 -2, 만료 없는 키 -1, 만료 예정 키는 남은 초 정수 반환.

4. 과제 요구사항 8번 결과 예시 100% 일치 검증 로그

터미널에서 python3 demo.py 또는 python3 main.py 실행 시 출력되는 실제 실행 결과입니다:

```
mini-redis> CONFIG SET maxmemory 30
OK
mini-redis> SET user:1 "Alice"
OK
mini-redis> SET user:2 "Bob"
OK
mini-redis> SET user:3 "Charlie"
OK
# maxmemory(30) 초과로 LRU(user:1) 제거 확인
mini-redis> GET user:1
(nil)
mini-redis> INFO memory
used_memory:22
maxmemory:30
evicted_keys:1
mini-redis> KEYS
1. "user:2"
2. "user:3"
mini-redis> EXPIRE user:2 3
(integer) 1
mini-redis> TTL user:2
(integer) 2
# (3초 경과 후)
mini-redis> GET user:2
(nil)
mini-redis> TTL user:2
(integer) -2

mini-redis> CONFIG SET maxmemory abc
(error) ERR value is not an integer or out of range
mini-redis> GET
(error) ERR wrong number of arguments for 'GET' command
mini-redis> HELLO
(error) ERR unknown command 'HELLO'
```

5. 단위 테스트 결과 (34개 테스트 100% 통과)

```
kangsikseo@KANGSIKs-MacBook-Pro miniredis % python3 -m unittest discover -s tests
..................................
----------------------------------------------------------------------
Ran 34 tests in 0.002s

OK
```

6. 보너스 과제 완주 내역 (선택 과제 5종 전체 구현)

1. 동적 배열 직접 구현 (DynamicArray): append, get, set, remove 구현 및 용량 초과 시 2배 자동 확장
2. 스택/큐/덱 조사 문서화: STACK_QUEUE_DEQUE.md 작성 (빅오 복잡도, 원리 및 활용 비교)
3. 이진 트리 4대 순회: BinaryTree 전위, 중위, 후위, 레벨 순회(BFS 큐) 구현
4. 이진 탐색 트리 (BST): BinarySearchTree O(log N) 삽입, 탐색 및 중위 순회 오름차순 정렬
5. Pub/Sub 기능: PubSubBroker 발행/구독 패턴 및 채널별 메시지 큐 구현

결론 및 깃허브 저장소 확인:

본 프로젝트의 모든 소스코드, 34개 단위 테스트, 상세 README 및 자동 데모 스크립트는 공식 GitHub 저장소에 안전하게 커밋 및 푸시되어 있습니다.

GitHub Repository: https://github.com/KANGSIK-SEO/miniredis